

```

1  /* =====
2      sketch.js – p5-Zeichnung für Bel-Ami v3
3
4      Der Scroll-Akt: preload/setup/draw plus die Zustandsvariablen, an denen die
5      zehn Module hängen. draw() leitet aus der Scrollposition alle Phasen ab und
6      ruft die Zeichenmodule in fester Reihenfolge.
7  ===== */
8
9  // --- Modulkapselung -----
10 // 39 von 67 Namen intern, 28 exportiert. Konvention: docs/architektur.md.
11 (function () {
12
13     let stage, heroText, begleitTexte, kapitelEinstiegsTexte;
14     let demoGruppenTexte; // die neun .begleittext mit data-demo-gruppe – ihre Fenster steuern auch die
15     // Stufen des Legendenaufbaus
16     let fotoHinweisText; // der .begleittext mit data-foto-hinweis – sein Fenster und sein Zielmarker
17     // steuern den Bedienhinweis an der Karte
18
19     // Text des Bedienhinweises am Fotomarker. Auf welchen Marker er zeigt und
20     // wann, steht am Begleittext in index.html – hier nur der Wortlaut.
21     const FOTO_HINWEIS_TEXT = 'Diese Punkte lassen sich anklicken, um ein historisches Foto zu sehen.';
22     let annotationBoxEl; // #annotationBox – trägt die Positionsklasse (pos-oben-links etc.), siehe
23     // annotationBoxPosition()
24     let schlusstextEl; // #schlusstext – Gegenstück zum Einstiegstext, blendet im Schlussakt ein
25     let kapitelEndeEl, kapitelEndeWeiterEl, kapitelEndeTextEl; // Kapitelende: Buttonpaar und der
26     // Hinweis darüber (nur Kapitel 1), siehe draw()
27     let projekttextEl, einblenderSchliessenEl; // #projekttext (dunkle Textfläche) und das
28     // Schliesskreuz, das sich beide Einblender teilen
29
30     // Nummer des folgenden Kapitels, oder null bei 18. Gilt für 02–18; Kapitel 1
31     // hat seinen eigenen Scroll-Akt.
32     function naechstesKapitel(nr) {
33         if (!nr) return null;
34         let ziel = String(parseInt(nr, 10) + 1).padStart(2, '0');
35         return kapitelHatEigeneAnsicht(ziel) ? ziel : null;
36     }
37
38     // Kapitel 02–18 öffnen per Klick, nicht per Scroll – daher kein
39     // data-von/data-bis, sondern ein Zeitfenster-Fade ab dem Klick.
40     let kapitelEinstiegsStartMillis = null;
41     const KAPITEL_EINSTIEG_FADE_MS = 800;
42
43     // Hält einen Klick beim DOM-Element und lässt ihn nicht zu p5 durch.
44
45     // ACHTUNG stopPropagation muss am mousedown hängen, nicht am click: p5 löst
46     // mousePressed schon beim mousedown aus, der click kommt erst danach. Am
47     // click abgefangen liefere der Klick trotzdem durch die Prüfungen in
48     // mousePressed – er schliesse etwa den Einblender, in dem gerade gelesen wird.
49     function haltKlickAuf(el, beiKlick) {
50         el.addEventListener('mousedown', ev => ev.stopPropagation());
51         if (beiKlick) el.addEventListener('click', beiKlick);
52     }
53
54     // Setzt die Einstiegstext-Uhr neu, gerufen von uebersichtsrouten.js beim
55     // Kapitelwechsel.
56     function starteKapitelEinstieg() {
57         kapitelEinstiegsStartMillis = millis();
58     }
59
60     // Der Einstiegstext blendet zeitbasiert ein, dann übernimmt das Scrollen:
61     // zwischen diesen Anteilen des Akts blendet er aus, erst danach die Route.
62     const KAPITEL_EINSTIEG_SCROLL_START = 0.015;
63     const KAPITEL_EINSTIEG_SCROLL_ENDE = 0.06;
64     // bgImage: Startseite und Schlusskarte. bgImage2: Übersichtsakt.
65     // ACHTUNG die beiden haben NICHT dieselbe Bbox – 568 m versetzt, deshalb

```

```

62 // startBbox und uebersichtBbox getrennt. Siehe docs/bugfix-log.md, Fix 2.
63 let bgImage, bgImage2, ch1Image;
64 let kartenMarkierungenEl;
65 let stationenData;
66 let kapitel03Data; // eigenes Datenset fürs Kapitel-3-Spine-Panel (Kartenausschnitt-Zoom)
67
68 // Erstentwurf-Datensätze für Kapitel 2, 4-18 (baue-kapitel-stationen.py).
69 // Zugriff nur über datenFuerKapitel(), auch aus vier anderen Modulen.
70 const WEITERE_KAPITEL_NUMMERN = ['02', '04', '05', '06', '07', '08', '09', '10', '11', '12', '13',
  '14', '15', '16', '17', '18'];
71 let weitereKapitelDaten = {}; // z.B. weitereKapitelDaten['04'].ortRuns
72 // Ortspunkte/Labels der Kapitel-1-Ansicht sind stillgelegt: dom-aufbau.js baut
73 // die Knoten weiter, draw() überspringt ihre Positionierung.
74
75 // ACHTUNG zum Wiedereinschalten reicht true nicht – zusätzlich muss
76 // .karten-markierung .label { display: none } in style.css fallen, sonst
77 // erscheinen nur Punkte ohne Beschriftung.
78 const KARTEN_MARKER_SICHTBAR = false;
79 let markierungsEintraege = [];
80 let stationsMarker = [];
81 let zwischenMarker = [];
82 let annotationText;
83 let annotationInner;
84 let annotationTag;
85 let annotationBar;
86 let scrollFortschritt, scrollFortschrittFuellung; // Fortschrittsleiste unten (Übersicht
  Scrolltelling-Hauptstrang) – ausgeblendet während einer Kapitel-Ansicht (siehe
  kapitelAnsichtsModus)
87 let kapitelRegister; // Kapitelregister rechts (inkl. Plan/Graph + Alle), sichtbar während eines
  Kapitel-Zooms
88 let kapitelRegisterEintraege = {}; // nr -> Eintrags-Element, fürs Aktiv-Highlighting in draw()
89 let planEintrag, graphEintrag; // "Plan"/"Graph"-Hälften oben im Register, fürs Aktiv-Highlighting
  in draw()
90 let modusZeile, leerzeile, alleEintrag; // Plan/Graph-Zeile + Abstandshalter + "Alle" – in der
  Übersicht (kein Kapitel gezoomt) blendet draw() modusZeile/leerzeile aus und markiert alleEintrag
  als aktiv
91
92 // Kapitel 1 endet, wo der Übersichtsakt beginnt: dort ist die Scrollposition
93 // festgehalten, die Überblickskarte kommt nur noch per Klick. Jeder Sprung
94 // dahinter löst die Klemme, ein Zurückscrollen in Kapitel 1 setzt sie neu.
95 let kapitel1Geklemmt = true;
96
97 // Gerufen von uebersichtsrouten.js aus den beiden Sprungzielen hinter
98 // Kapitel 1 ("Übersicht"/"Alle" und jeder Kapitel-Zoom).
99 function loeseKapitel1Klemme() {
100   kapitel1Geklemmt = false;
101 }
102
103 // Erklärungs-Ebene über der laufenden Ansicht, geöffnet per Klick aufs
104 // Kreisgrafik-Icon oben rechts. Jeder weitere Klick schliesst sie wieder.
105 let kreisErklaerungOffen = false;
106
107 // Der Projekttext-Einblender hat zwei Wege hinein: am Ende der Route geht er
108 // von selbst auf, und das zweite Icon holt ihn jederzeit zurück. Deshalb zwei
109 // Merker statt einem – sonst liesse sich der automatische nicht wegklicken,
110 // ohne den per Icon geöffneten mitzuschliessen.
111 let projekttextPerIkon = false; // per Icon geholt, bleibt bis zum nächsten Klick
112 let projekttextWeggeklickt = false; // der automatische wurde von Hand geschlossen
113 let projekttextOffen = false; // je Frame aus den beiden abgeleitet
114 const PROJEKTTEXT_SCHLEIER = '#212B2E';
115 const PROJEKTTEXT_SCHLEIER_ALPHA = 0.94;
116
117 // Heller Schleier unter dem Legendenaufbau. Derselbe Ton wie die
118 // Erklärungs-Ebene in kreisgrafik.js: die helle Karte bleibt darunter als
119 // Karte erkennbar, tritt aber hinter die Legende zurück.
120 const LEGENDE_SCHLEIER = '#E2E6E1';

```

```

121 const LEGENDE_SCHLEIER_ALPHA = 0.8;
122
123 // Zu heisst je nach Weg etwas anderes: den per Icon geholten einfach wieder
124 // weg, den automatischen für diesen Durchlauf abhaken.
125 function schliesseProjekttext() {
126     if (projekttextPerIkon) projekttextPerIkon = false;
127     else projekttextWeggeklickt = true;
128 }
129
130 // Zwei Modi je Kapitel-Ansicht: 'karte' (Ausschnitt und Route) und 'grafik'
131 // (horizontale Spine mit Play). Umschalten über "Plan"/"Graph" im Menübalken.
132 let kapitelAnsichtsModus = 'karte';
133
134 // Nur das Setzen liegt hier; den restlichen Zustand setzt jedes Modul selbst
135 // zurück (setzeGrafikZurueck, starteKapitelEinstieg).
136 function setzeAnsichtsModus(modus) {
137     kapitelAnsichtsModus = modus;
138 }
139 // Zoomstand des Kapitel-1-Ausschnitts (0..1), je Frame in draw() gesetzt.
140 // kreisgrafik.js blendet daran das Label des Routen-Startpunkts ein.
141 let kapitel1ZoomAmount = 0;
142 let grafikPlayButton;
143
144 // --- Übersichtsrouten (Kapitel 02–18, nur in der letzten, rausgezoomten Ansicht) ---
145 let uebersichtsRouten = {};
146
147 // Kapitel mit eigenem Kartenausschnitt (bilder-karten/kapitelXX-*): alle
148 // ausser 01, das sein eigenes System hat. Die Ausschlussliste ist leer.
149
150 // vAnchor/hAnchor verschieben den sichtbaren Ausschnitt im Kapitelbild
151 // (0 = oben/links, 1 = unten/rechts, 0.5 = Default), siehe coverCrop.
152 const OHNE_EIGENEN_KARTENAUSSCHNITT = [];
153 let kapitelKarten = {
154     '02': { bild: null, bboxRaw: null },
155     '03': { bild: null, bboxRaw: null, vAnchor: 0.15 },
156     '04': { bild: null, bboxRaw: null },
157     '05': { bild: null, bboxRaw: null },
158     '06': { bild: null, bboxRaw: null },
159     '07': { bild: null, bboxRaw: null },
160     '08': { bild: null, bboxRaw: null },
161     '09': { bild: null, bboxRaw: null },
162     '10': { bild: null, bboxRaw: null },
163     '11': { bild: null, bboxRaw: null },
164     '12': { bild: null, bboxRaw: null },
165     '13': { bild: null, bboxRaw: null },
166     '14': { bild: null, bboxRaw: null },
167     '15': { bild: null, bboxRaw: null },
168     '16': { bild: null, bboxRaw: null },
169     '17': { bild: null, bboxRaw: null },
170     '18': { bild: null, bboxRaw: null },
171 };
172 let letzterZoomKapitel = null; // bleibt waehrend des Ausblendens gesetzt, siehe draw()
173
174 // Datensatz zu einer Kapitelnummer. Kapitel 3 hat eine eigene Variable,
175 // alle anderen liegen in weitereKapitelDaten.
176 function datenFuerKapitel(nr) {
177     return nr === '03' ? kapitel03Data : weitereKapitelDaten[nr];
178 }
179
180 // Hat dieses Kapitel eine öffenbare Ansicht? Eigener Kartenausschnitt oder
181 // zumindest ein Spine-Panel. sketch.js hält das Kapitelinventar.
182
183 // ACHTUNG !!kapitelKarten[nr] prüft die Inventarzugehörigkeit, nicht ob das
184 // Bild geladen ist – .bild füllt erst preload(). Die ODER-Klausel greift
185 // heute nie: beide Listen führen dieselben 17 Kapitel. Zwei handgepflegte
186 // Listen für dasselbe sind der eigentliche Mangel, ungelöst.

```

```

187 function kapitelHatEigeneAnsicht(nr) {
188   return !!kapitelKarten[nr] || KAPITEL_MIT_SPINE_PANEL.has(nr);
189 }
190
191 function preload() {
192   bgImage = loadImage('bilder-karten/paris-startkarte-web.png');
193   bgImage2 = loadImage('bilder-karten/paris-ueberblickkarte-web.png');
194   ch1Image = loadImage('bilder-karten/kapitel01-qgis-karte-web.png');
195
196   const kapitelDatenDateien = [
197     { nr: '01', ziel: 'stationenData' },
198     { nr: '03', ziel: 'kapitel03Data' },
199     ...WEITERE_KAPITEL_NUMMERN.map(nr => ({ nr, ziel: nr }))
200   ];
201
202   kapitelDatenDateien.forEach(({ nr, ziel }) => {
203     const datei = `kapitel${nr}-stationen.json`;
204     if (ziel === 'stationenData') stationenData = loadJSON(datei);
205     else if (ziel === 'kapitel03Data') kapitel03Data = loadJSON(datei);
206     else weitereKapitelDaten[ziel] = loadJSON(datei);
207   });
208
209   fotoMarkerListe = loadJSON('fotomarker.json');
210   uebersichtsRouten = loadJSON('kapitel-routen-uebersicht.json');
211
212   Object.keys(kapitelKarten).forEach(nr => {
213     if (OHNE_EIGENEN_KARTENAUSSCHNITT.includes(nr)) return;
214     kapitelKarten[nr].bild = loadImage(`bilder-karten/kapitel${nr}-karte.png`);
215     kapitelKarten[nr].bboxRaw = loadJSON(`bilder-karten/kapitel${nr}-bbox.json`);
216   });
217 }
218
219 function bereinigeEingangsdaten() {
220   [stationenData, kapitel03Data, ...WEITERE_KAPITEL_NUMMERN.map(nr => weitereKapitelDaten[nr])]
221     .filter(Boolean)
222     .forEach(daten => {
223       bereinigeStationenDaten(daten);
224     });
225
226   fotoMarkerListe = bereinigeFotoMarker(fotoMarkerListe);
227   uebersichtsRouten = bereinigeUebersichtsrouten(uebersichtsRouten);
228 }
229
230 function setup() {
231   bereinigeEingangsdaten();
232
233   stage = document.getElementById('scrollyStage');
234   heroText = document.querySelectorAll('h1, h2, .lead, .scroll-hinweis');
235   begleitTexte = document.querySelectorAll('.begleittext');
236   demoGruppenTexte = [...begleitTexte].filter(el => el.dataset.demoGruppe)
237     .sort((a, b) => a.dataset.demoGruppe - b.dataset.demoGruppe);
238   // Ein Begleittext trägt den Hinweis; sein data-Wert nennt den Fotomarker.
239   fotoHinweisText = [...begleitTexte].find(el => el.dataset.fotoHinweis);
240   kapitelEinstiegsTexte = document.querySelectorAll('.kapitel-einstiegstext');
241
242   kartenMarkierungenEl = document.getElementById('kartenMarkierungen');
243   kapitelEndeEl = document.getElementById('kapitelEnde');
244   kapitelEndeTextEl = document.getElementById('kapitelEndeText');
245   kapitelEndeWeiterEl = document.getElementById('kapitelEndeWeiter');
246   haltKlickAuf(kapitelEndeWeiterEl, () => {
247     // Kapitel 1 ist kein Zoom, hat aber dasselbe Kapitelende.
248     let ziel = naechstesKapitel(zoomedKapitel || '01');
249     if (ziel) springeZuKapitelZoom(ziel);
250   });
251   haltKlickAuf(document.getElementById('kapitelEndeUebersicht'), springeZurUebersicht);
252

```

```

253 projekttextEl = document.getElementById('projekttext');
254 // Klicks im Text bleiben im Text – daneben schliesst p5 den Einblender.
255 haltKlickAuf(document.getElementById('projekttextInner'));
256 einblenderSchliessenEl = document.getElementById('einblenderSchliessen');
257 haltKlickAuf(einblenderSchliessenEl, () => {
258     if (projekttextOffen) schliesseProjekttext();
259     kreisErklaerungOffen = false;
260 });
261 annotationBoxEl = document.getElementById('annotationBox');
262 schlusstextEl = document.getElementById('schlusstext');
263 annotationText = document.getElementById('annotationText');
264 annotationInner = document.getElementById('annotationInner');
265 annotationTag = document.getElementById('annotationTag');
266 annotationBar = document.getElementById('annotationBar');
267 kapitelRegister = document.getElementById('kapitelRegister');
268 scrollFortschritt = document.getElementById('scrollFortschritt');
269 grafikPlayButton = document.getElementById('grafikPlayButton');
270 grafikPlayButton.addEventListener('click', toggleGrafikPlay);
271
272 fotoPopup = document.getElementById('fotoPopup');
273 fotoPopupTitel = document.getElementById('fotoPopupTitel');
274 fotoPopupPlz = document.getElementById('fotoPopupPlz');
275 fotoPopupBild = document.getElementById('fotoPopupBild');
276 fotoPopupBeschreibung = document.getElementById('fotoPopupBeschreibung');
277 scrollFortschrittFuellung = document.getElementById('scrollFortschrittFuellung');
278 document.getElementById('fotoPopupClose').addEventListener('click', schliesseFotoPopup);
279 fotoPopup.addEventListener('click', e => { if (e.target === fotoPopup) schliesseFotoPopup(); });
280 document.addEventListener('keydown', e => {
281     if (e.key === 'Escape') {
282         kreisErklaerungOffen = false;
283         schliesseProjekttext();
284         schliesseFotoPopup();
285         schliesseKapitelZoom();
286     }
287 });
288
289 let cnv = createCanvas(stage.offsetWidth, stage.offsetHeight);
290 cnv.parent('scrollyStage');
291
292 baueKartenMarkierungen();
293 // ACHTUNG Destrukturierungs-ZUWEISUNG ohne const/let: die Handles sind oben
294 // auf Modulebene deklariert. Ein `const { ... } =` legte funktionslokale
295 // Konstanten an, die Modulvariablen blieben undefined – der Menübalken
296 // fiel still aus.
297 //
298 // Vorher schrieb dom-aufbau.js diese fünf direkt von aussen; jetzt baut es
299 // und gibt zurück (siehe docs/best-practices-review.md, "Gruppe B").
300 ({ modusZeile, planEintrag, graphEintrag, leerzeile, alleEintrag } = baueKapitelRegister());
301 baueStationsMarker();
302 baueZwischenMarker();
303 }
304
305 function windowResized() {
306     resizeCanvas(stage.offsetWidth, stage.offsetHeight);
307 }
308
309 // Hält die Scrollposition an einer Marke fest und gibt sie zurück. Nach oben
310 // bleibt sie frei – der Weg zurück.
311 function klemmeScroll(marke) {
312     let trackEl = document.querySelector('.scroll-track');
313     window.scrollTo(0, trackEl.offsetHeight * marke);
314     return marke;
315 }
316
317 function getScrollProgress() {
318     let trackEl = document.querySelector('.scroll-track');

```

```

319     return constrain(window.scrollY / trackEl.offsetHeight, 0, 1);
320 }
321
322 // Deckkraft eines scrollgebundenen Fensters: Rampe rein, Plateau, Rampe raus.
323 // Rampe höchstens 35% des Fensters, sonst erreicht ein kurzes Fenster nie
324 // volle Deckkraft. Auch die Beschriftungen am Demo-Kreis hängen daran.
325 function fadeDauerFuer(von, bis) {
326     let fadeDauerMax = 0.142857; // 0.2 auf die verlängerte Scrollstrecke umskaliert (2200/3080)
327     return Math.min(fadeDauerMax, (bis - von) * 0.35);
328 }
329
330 function begleittextDeckkraft(progress, von, bis) {
331     let fadeDauer = fadeDauerFuer(von, bis);
332     return constrain(Math.min(
333         map(progress, von, von + fadeDauer, 0, 1),
334         map(progress, bis - fadeDauer, bis, 1, 0)), 0, 1);
335 }
336
337 // Wie begleittextDeckkraft, aber ohne das Ausblenden: der Legendenaufbau ist
338 // kumulativ wie im PDF – was einmal steht, bleibt stehen, bis der Schleier am
339 // Ende alles zusammen mitnimmt. Gleiche Einblenddauer, damit ein Schritt und
340 // sein Erklärtext gemeinsam kommen.
341 function legendenSchrittDeckkraft(progress, von, bis) {
342     return constrain(map(progress, von, von + fadeDauerFuer(von, bis), 0, 1), 0, 1);
343 }
344
345 // -----
346 // draw()
347 // -----
348
349 function draw() {
350     background(220);
351
352     // letzterZoomKapitel bleibt nach dem Schliessen gesetzt, damit das
353     // Spine-Panel beim Ausblenden noch die richtigen Daten zeigt.
354     if (zoomedKapitel) letzterZoomKapitel = zoomedKapitel;
355     stelleSpineDatenBereit(letzterZoomKapitel);
356     let targetCrop = coverCrop(ch1Image.width, ch1Image.height);
357     let targetBbox = cropToBbox(targetCrop, ch1ImgBbox, ch1Image.width, ch1Image.height);
358
359     let progress = getScrollProgress();
360     // Ein offenes Kapitel endet mit seiner letzten Annotation: Scrollposition
361     // wird nach unten festgehalten, nach oben bleibt sie frei – der Weg zurück.
362     if (zoomedKapitel && progress > SCROLL_MEILENSTEINE.uebersichtRoutenEnd) {
363         progress = klemmeScroll(SCROLL_MEILENSTEINE.uebersichtRoutenEnd);
364     }
365     // Kapitel 1 endet genauso, nur liegt seine Grenze am Anfang des
366     // Übersichtsakts. Zurück in Kapitel 1 gescrollt, steht die Klemme wieder.
367     if (progress < SCROLL_MEILENSTEINE.uebersichtRoutenStart) kapitel1Geklemmt = true;
368     if (!zoomedKapitel && kapitel1Geklemmt && progress > SCROLL_MEILENSTEINE.uebersichtRoutenStart) {
369         progress = klemmeScroll(SCROLL_MEILENSTEINE.uebersichtRoutenStart);
370     }
371     scrollFortschrittFuellung.style.width = (progress * 100) + '%';
372
373     // Startkarte blendet vor dem Zoom auf die helle Überblickskarte über.
374     // Im Schlussakt kehrt die STARTkarte wieder, mit eigener Georeferenz.
375     let imStartkarteAkt = progress >= SCROLL_MEILENSTEINE.startkarteStart;
376     let kartenwechsel = imStartkarteAkt ? 0 : constrain(map(progress,
377         SCROLL_MEILENSTEINE.kartenwechselStart, SCROLL_MEILENSTEINE.kartenwechselEnd, 0, 1), 0, 1);
378     let fullCrop = coverCrop(bgImage.width, bgImage.height, 0.5, 0.5, 0); // grosse Karte bleibt
379     // zentriert, unabhängig von mapOffsetX
380     // ACHTUNG der Ausschnitt muss in BEIDE Karten passen: sonst klemmt
381     // bboxToImgCrop ihn an der Bildkante und streckt das Bild dabei um 2.5%.
382     // Deshalb in die Schnittmenge beider Georeferenzen eingepasst.
383     let fullBbox = passeBboxInRahmen(
384         cropToBbox(fullCrop, startBbox, bgImage.width, bgImage.height), UEBERSICHT_SCHNITT_BBBOX);

```



```

384
385     let zoomAmount = constrain(map(progress, SCROLL_MEILENSTEINE.zoomStart,
SCROLL_MEILENSTEINE.zoomEnd, 0, 1), 0, 1);
386     // Solange Kapitel 1s Klemme steht, liegt die Karte in Kapitel 1; ein Klick
387     // hinter das Kapitel schaltet auf die Überblickskarte um. Einen Rauszoom-Akt
388     // gibt es nicht mehr, und der Sprung ist ohnehin ein Schnitt – deshalb
389     // schaltet der Wert hart um, statt zu rampen. Route und Kreise bleiben
390     // sichtbar: routeAmount hängt nicht am Zoom.
391     let zoomOutAmount = kapitel1Geklemmt ? 0 : 1;
392     zoomAmount *= (1 - zoomOutAmount);
393     kapitel1ZoomAmount = zoomAmount;
394
395     // "In einer Kapitel-Ansicht": ein gezoomtes Kapitel 02–18 ODER Kapitel 1s
396     // eigener Ausschnitt. Grafik-Ansicht zusätzlich nur bei Modus 'grafik'.
397     let inKapitel1Kartenausschnitt = !zoomedKapitel && zoomAmount > 0.5;
398     let inKapitelAnsicht = !zoomedKapitel || inKapitel1Kartenausschnitt;
399     let inKapitelGrafikAnsicht = inKapitelAnsicht && kapitelAnsichtsModus === 'grafik';
400
401     // Gemeinsamer Offset aller Overlays, 0 bis mapOffsetX mit dem Zoom –
402     // sonst sitzen sie beim Zoomen neben der sichtbaren Karte.
403     let kartenOffsetX = lerp(0, mapOffsetX, zoomAmount);
404     let kartenOffsetY = lerp(0, mapOffsetY, zoomAmount);
405
406     // Schon hier berechnet, weil activeBbox unten davon abhängt. Hochscrollen
407     // vor den Aktanfang schliesst einen offenen Kapitel-Zoom wieder.
408     let uebersichtRoutenFortschritt = constrain(map(progress,
SCROLL_MEILENSTEINE.uebersichtRoutenStart, SCROLL_MEILENSTEINE.uebersichtRoutenEnd, 0, 1), 0, 1);
409     if (zoomedKapitel && uebersichtRoutenFortschritt <= 0) schliesseKapitelZoom(); // zurückgescrollt
410
411     // Schlussakt Ortsveränderung: ein Fortschritt 0..1 über den ganzen Akt,
412     // aus dem die Phasen OV_*/SK_* abgeleitet werden.
413     let ovFortschritt = constrain(map(progress, SCROLL_MEILENSTEINE.kreisVergleichStart,
SCROLL_MEILENSTEINE.kreisVergleichEnd, 0, 1), 0, 1);
414     let skFortschritt = constrain(map(progress, SCROLL_MEILENSTEINE.startkarteStart, 1, 0, 1), 0, 1);
415     let skEinblenden = ovPhase(skFortschritt, SK_EINBLENDEN);
416     let skRauszoom = ovPhase(skFortschritt, SK_RAUSZOOM);
417     if (schlusstextEl) schlusstextEl.style.opacity = ovPhase(skFortschritt, SK_TEXT);
418     let kreisVergleichMapFade = ovPhase(ovFortschritt, OV_KARTE_AUS);
419     // Ein offener Kapitel-Zoom darf nicht in diesen Akt hinübergescrollt werden.
420     if (zoomedKapitel && kreisVergleichMapFade > 0) schliesseKapitelZoom();
421
422     // Kapitel-Zoom öffnet sofort mit voller Route, nur zeitlich eingeblendet.
423     aktualisiereKapitelZoom();
424
425     let activeBbox = {
426       west: lerp(fullBbox.west, targetBbox.west, zoomAmount),
427       east: lerp(fullBbox.east, targetBbox.east, zoomAmount),
428       south: lerp(fullBbox.south, targetBbox.south, zoomAmount),
429       north: lerp(fullBbox.north, targetBbox.north, zoomAmount),
430     };
431
432     // Schlusszoom auf die sieben Orte, erst nach dem Schrumpfen der Linien –
433     // so rücken sie weit genug auseinander für ihre Kreise.
434     let ovZoom = ovPhase(ovFortschritt, OV_ZOOM);
435     if (ovZoom > 0) {
436       let ziel = ovZoomBbox();
437       if (ziel) {
438         activeBbox = {
439           west: lerp(activeBbox.west, ziel.west, ovZoom),
440           east: lerp(activeBbox.east, ziel.east, ovZoom),
441           south: lerp(activeBbox.south, ziel.south, ovZoom),
442           north: lerp(activeBbox.north, ziel.north, ovZoom),
443         };
444       }
445     }
446

```

```

447 // Und wieder heraus: aus dem Ausschnitt der sieben Orte zurück auf die
448 // ganze Startkarte, nachdem diese eingeblendet hat.
449 if (skRauszoom > 0) {
450   activeBbox = {
451     west: lerp(activeBbox.west, fullBbox.west, skRauszoom),
452     east: lerp(activeBbox.east, fullBbox.east, skRauszoom),
453     south: lerp(activeBbox.south, fullBbox.south, skRauszoom),
454     north: lerp(activeBbox.north, fullBbox.north, skRauszoom),
455   };
456 }
457
458 // Kapitel-Zoom (Klick auf «03» etc.) – zoomt von der Gesamtkarte weiter in
459 // den eigenen Kartenausschnitt des Kapitels, genau wie oben bgImage→ch1Image.
460 let kapitelCrop = null;
461 // Ausserhalb des Blocks: die Annotationsbox unten braucht dieselbe
462 // unaniimierte Ziel-Bbox, sonst wandert ihre Position beim Reinzoomen.
463 let kapitelTargetBbox = null;
464 if (zoomedKapitel && kapitelKarten[zoomedKapitel] && kapitelKarten[zoomedKapitel].bild
465   && kapitelKarten[zoomedKapitel].bild.width && kapitelKarten[zoomedKapitel].bboxRaw) {
466   let k = kapitelKarten[zoomedKapitel];
467   kapitelCrop = coverCrop(k.bild.width, k.bild.height, k.vAnchor ?? 0.5, k.hAnchor ?? 0.5);
468   kapitelTargetBbox = cropToBbox(kapitelCrop, k.bboxRaw, k.bild.width, k.bild.height);
469   activeBbox = {
470     west: lerp(activeBbox.west, kapitelTargetBbox.west, kapitelZoomAmount),
471     east: lerp(activeBbox.east, kapitelTargetBbox.east, kapitelZoomAmount),
472     south: lerp(activeBbox.south, kapitelTargetBbox.south, kapitelZoomAmount),
473     north: lerp(activeBbox.north, kapitelTargetBbox.north, kapitelZoomAmount),
474   };
475 }
476
477 // ch1Image zoomt nicht mit, es blendet an seiner festen Zielposition ein.
478 // Ein dynamischer Ausschnitt würde geklemmt und dabei verzerrt.
479 let ch1Crop = targetCrop;
480
481 // (1 – kreisVergleichMapFade) blendet die Karte im Ortsveränderungs-Akt aus;
482 // skEinblenden holt sie im allerletzten Akt zurück.
483 let kartenAlpha = 255 * (1 – zoomAmount) * Math.max(1 – kreisVergleichMapFade, skEinblenden);
484 // Beide Karten auf dieselbe activeBbox, je aus der eigenen Georeferenz –
485 // so liegen sie im Crossfade deckungsgleich übereinander.
486 if (kartenwechsel < 1) {
487   let startCrop = bboxToImgCrop(activeBbox, startBbox, bgImage.width, bgImage.height);
488   tint(255, kartenAlpha);
489   image(bgImage, 0, 0, width, height, startCrop.x, startCrop.y, startCrop.w, startCrop.h);
490 }
491 if (kartenwechsel > 0) {
492   let uebersichtCrop = bboxToImgCrop(activeBbox, uebersichtBbox, bgImage2.width, bgImage2.height);
493   tint(255, kartenAlpha * kartenwechsel);
494   image(bgImage2, 0, 0, width, height, uebersichtCrop.x, uebersichtCrop.y, uebersichtCrop.w,
uebersichtCrop.h);
495 }
496 tint(255, 255 * zoomAmount * (1 – kreisVergleichMapFade));
497 image(ch1Image, mapOffsetX, mapOffsetY, width – mapOffsetX, height, ch1Crop.x, ch1Crop.y,
ch1Crop.w, ch1Crop.h);
498 noTint();
499
500 if (kapitelCrop && kapitelZoomAmount > 0.001) {
501   let k = kapitelKarten[zoomedKapitel];
502   tint(255, 255 * kapitelZoomAmount * (1 – kreisVergleichMapFade));
503   image(k.bild, mapOffsetX, mapOffsetY, width – mapOffsetX, height, kapitelCrop.x, kapitelCrop.y,
kapitelCrop.w, kapitelCrop.h);
504   noTint();
505 }
506
507 let massstabOffsetX = (kapitelCrop && kapitelZoomAmount > 0.5) ? mapOffsetX : kartenOffsetX;
508 zeichneMassstabsleiste(activeBbox, massstabOffsetX, 1 – kreisVergleichMapFade);
509 // Windrose stillgelegt; der Platz oben rechts gehört jetzt dem

```



```

510 // Kreisgrafik-Icon. Zum Wiedereinschalten diese Zeile entkommentieren.
511 // zeichneWindrose(width - 90, 150, 50, 1 - kreisVergleichMapFade);
512
513 let routeAmount = constrain(map(progress, SCROLL_MEILENSTEINE.routeStart,
SCROLL_MEILENSTEINE.routeEnd, 0, 1), 0, 1);
514
515 let annListe = stationenData.annotationen;
516 let annIndex = Math.min(annListe.length - 1, Math.floor(routeAmount * annListe.length));
517 let punktIndex = annListe[annIndex].revealIndex;
518 if (annListe[annIndex].vorRoutenstart) punktIndex = 0;
519
520 let endStation = stationenData.route.filter(s => s.routeEndsHere).pop()
521 || stationenData.route[stationenData.route.length - 1];
522 let liniIndex = Math.min(punktIndex, endStation.revealIndex);
523
524 // Übersichtsrouten 02-18, zuerst gezeichnet – Kapitel 1s Route liegt darüber.
525 let aktuelleAnnotationZoom = null;
526 // Die Routen bleiben stehen, damit die Ortspunkte auf ihnen landen; erst
527 // der Zoom lässt sie verschwinden, im Schlussakt kommen sie zurück.
528
529 // Unbedingt aufrufen, auch bei Fortschritt 0: zeichneUebersichtsrouten()
530 // setzt dann selbst kapitelHover und den Cursor zurück.
531 let routenSichtbar = Math.max((1 - 0.45 * kreisVergleichMapFade) * (1 - ovZoom), skRauszoom);
532 let routenAlpha = 180 * routenSichtbar;
533 let uebersichtRoutenErgebnis = zeichneUebersichtsrouten(activeBbox, routenAlpha,
uebersichtRoutenFortschritt);
534 aktuelleAnnotationZoom = uebersichtRoutenErgebnis.aktuelleAnnotationZoom;
535
536 // ACHTUNG Sperre nötig: bei gezoomtem Kapitel zeigt activeBbox dessen Bbox,
537 // nicht Kapitel 1s Gegend. Ohne sie liefere Kapitel 1s Route quer über den
538 // fremden Kartenausschnitt.
539 if (!zoomedKapitel) {
540 // Strichstärke 10 -> 2 beim Rauszoomen, wie die Übersichtsrouten. Auch
541 // dasselbe Ausblenden, sonst verschwände sie vor den anderen.
542 zeichneRoute(stationenData.routenPunkte, liniIndex, activeBbox, lerp(3, 2, zoomOutAmount),
kartenOffsetX, kartenOffsetY,
543 Math.max((1 - 0.45 * kreisVergleichMapFade) * (1 - ovZoom), skRauszoom));
544 // Kreisgrafik (Karte) in der letzten Ansicht (Rauszoomen) für den Moment
545 // ausgeblendet – Route/Spine bleiben davon unberührt sichtbar.
546 if (zoomOutAmount <= 0) {
547 zeichneKreiseOrtRuns(punktIndex, annIndex, activeBbox, kartenOffsetX, kartenOffsetY);
548 }
549 }
550
551 // Schlussakt, siehe ortsveraenderung.js.
552 if (ovFortschritt > 0 && !zoomedKapitel) {
553 zeichneOrtsveraenderung(activeBbox, ovFortschritt, 255 * (1 - skRauszoom), 1 - skEinblenden);
554 }
555
556 // Graph-Ansicht deckt Karte, Route und Kreise dieses Frames vollständig ab
557 // (zeichneSpineHorizontal, aktualisiereGrafikFortschritt).
558 if (inKapitelGrafikAnsicht) {
559 background(226, 230, 225); // #E2E6E1
560 // Die Karte samt ihren Kreisen ist damit übermalt; die Spine meldet
561 // gleich ihre eigenen an.
562 vergissGezeichneteKreise();
563 let grafikEintraege = spineEintraegeFuer(zoomedKapitel);
564 let grafikDaten = zoomedKapitel ? datenFuerKapitel(zoomedKapitel) : stationenData;
565 aktualisiereGrafikFortschritt();
566 zeichneSpineHorizontal(grafikEintraege || [], grafikFortschritt, grafikDaten);
567 }
568
569 // Kapitel 1 läuft über routeAmount/annIndex, ein gezoomtes Kapitel über
570 // aktuelleAnnotationZoom. Beide schliessen sich aus.
571 let aktuelleAnnotation = !zoomedKapitel
572 ? ((routeAmount > 0 && zoomOutAmount <= 0) ? annListe[annIndex] : null)

```

```

573 : (kapitelZoomAmount > 0.5 ? aktuelleAnnotationZoom : null);
574 // Ziel-Bbox des Kapitels, nicht die animierte activeBbox – sonst wandert
575 // die Positionswahl beim Reinzoomen.
576 let positionKapitel = zoomedKapitel || '01';
577 let positionBbox = zoomedKapitel ? kapitelTargetBbox : targetBbox;
578 let positionDaten = zoomedKapitel ? datenFuerKapitel(zoomedKapitel) : stationenData;
579 if (annotationBoxEl && positionBbox && positionDaten && positionDaten.ortRuns) {
580     let position = annotationBoxPosition(positionKapitel, positionDaten, positionBbox);
581     ANNOTATION_BOX_POSITIONEN.forEach(p => annotationBoxEl.classList.toggle('pos-' + p, p ===
position));
582 }
583
584 // Die Icons blenden erst im Schlussakt wieder aus; ohne sie gibt es auch
585 // keine Einblender. Steht hier oben, weil das Kapitelende darauf aufbaut.
586 let demoAlpha = progress < SCROLL_MEILENSTEINE.kartenwechselEnd ? 0 : 1 - skEinblenden;
587
588 // Der Projekttext geht am Ende der Route von selbst auf und bleibt, bis er
589 // weggeklickt oder durchgescrollt ist. Zurückgescrollt zählt als neuer
590 // Durchlauf, dann geht er wieder auf.
591 let imProjekttextFenster = !zoomedKapitel
592     && progress >= SCROLL_MEILENSTEINE.routeEnd
593     && progress < SCROLL_MEILENSTEINE.kapitelEndeStart;
594 if (progress < SCROLL_MEILENSTEINE.routeEnd) projekttextWeggeklickt = false;
595 // Ohne Icons keine Einblender: auf Start- und Schlusskarte gibt es beide nicht.
596 if (demoAlpha <= 0.01) { kreisErklaerungOffen = false; projekttextPerIkon = false; }
597 projekttextOffen = projekttextPerIkon || (imProjekttextFenster && !projekttextWeggeklickt);
598
599 // Kapitelende, in jedem Kapitel: die beiden Klickziele. In 02–18 steht der
600 // Scroll dort geklemmt, Kapitel 1 hat dafür seine eigene Strecke zwischen
601 // Projekttext und Klemme.
602
603 // Nicht an eine Scrollmarke allein gebunden, sondern an "der Projekttext ist
604 // zu": sonst zeigte der X-Weg die Kartenansicht ohne Hinweis und Buttons,
605 // bis man bis zur Klemme durchgescrollt hätte.
606 let kapitel1AmEnde = !zoomedKapitel && kapitel1Geklemmt && !projekttextOffen
607     && progress >= SCROLL_MEILENSTEINE.routeEnd;
608 let kapitelLokalerFortschritt = constrain(
609     map(uebersichtRoutenFortschritt, KAPITEL_EINSTIEG_SCROLL_ENDE, 1, 0, 1), 0, 1);
610 // Schwelle aus der Annotationszahl, nicht fest: dieselbe Rechnung wie
611 // annIndexZoom. Ein fester Wert läge je nach Kapittelänge daneben.
612 let endDaten = datenFuerKapitel(zoomedKapitel);
613 let anzahl = endDaten && endDaten.annotations ? endDaten.annotations.length : 0;
614 let zoomAmEnde = !!zoomedKapitel && kapitelZoomAmount > 0.5
615     && anzahl > 0 && kapitelLokalerFortschritt >= (anzahl - 1) / anzahl;
616 // Nur in der Kartenansicht – in der Graph-Ansicht sitzt dort der Play-Button.
617 // Ein Wert für beide Leser: das Buttonpaar und die Annotationsbox unten.
618 let amKapitelEnde = (kapitel1AmEnde || zoomAmEnde) && kapitelAnsichtsModus === 'karte';
619 if (kapitelEndeEl) {
620     kapitelEndeEl.classList.toggle('sichtbar', amKapitelEnde);
621     // Kapitel 18 hat keinen Nachfolger, dort bleibt nur "Übersicht".
622     kapitelEndeWeiterEl.classList.toggle('versteckt', !naechstesKapitel(zoomedKapitel || '01'));
623 }
624 // Der Hinweis darüber gilt nur für Kapitel 1. Er hängt an derselben Klasse
625 // wie das Buttonpaar statt an einem eigenen Scroll-Fenster – beide kommen
626 // und gehen zusammen.
627 if (kapitelEndeTextEl) kapitelEndeTextEl.classList.toggle('sichtbar', kapitel1AmEnde);
628
629 // Am Kapitelende schweigt die Annotationsbox. Sie lüde zum Weiterlesen ein,
630 // während Hinweis und Buttons daneben das Kapitel abschliessen – zwei
631 // Signale, die sich widersprechen.
632 if (amKapitelEnde) aktuelleAnnotation = null;
633
634 if (aktuelleAnnotation) {
635     annotationText.textContent = '«' + aktuelleAnnotation.text + '»';
636     annotationText.style.opacity = 1;
637     annotationInner.style.opacity = 1;

```

```

638     annotationInner.style.background = 'rgba(226, 230, 225, 0.85)';
639     let catColor = CATEGORY_COLORS[aktuelleAnnotation.category] || ROUTE_COLOR;
640     let fwertColor = FWERT_COLOR;
641     annotationBar.style.background = aktuelleAnnotation.hasFwert
642       ? `linear-gradient(90deg, ${catColor}, ${fwertColor})`
643       : catColor;
644     annotationTag.textContent = CATEGORY_LABELS[aktuelleAnnotation.category] || '';
645   } else {
646     annotationText.style.opacity = 0;
647     annotationInner.style.opacity = 0;
648     annotationTag.textContent = '';
649   }
650
651   // Kapitelregister: in jeder Kapitel-Ansicht und zusätzlich in der Übersicht,
652   // damit man von dort direkt in ein Kapitel springen kann. Im Schlussakt nicht.
653   let inUebersichtRouten = uebersichtRoutenFortschritt > 0 && !zoomedKapitel &&
kreisVergleichMapFade <= 0;
654   kapitelRegister.classList.toggle('sichtbar', inKapitelAnsicht || inUebersichtRouten);
655
656   // Plan/Graph nur in einer echten Kapitel-Ansicht; in der Übersicht ist
657   // "Alle" der aktive Eintrag.
658   modusZeile.classList.toggle('versteckt', !inKapitelAnsicht);
659   leerzeile.classList.toggle('versteckt', !inKapitelAnsicht);
660   alleEintrag.classList.toggle('aktiv', inUebersichtRouten);
661   if (inKapitelAnsicht) {
662     planEintrag.classList.toggle('aktiv', kapitelAnsichtsModus === 'karte');
663     graphEintrag.classList.toggle('aktiv', kapitelAnsichtsModus === 'grafik');
664     Object.entries(kapitelRegisterEintraege).forEach(([nr, eintrag]) => {
665       eintrag.classList.toggle('aktiv', zoomedKapitel ? nr === zoomedKapitel : nr === '01');
666     });
667   } else if (inUebersichtRouten) {
668     // Kein Kapitel aktiv, sonst bliebe eine veraltete Hervorhebung stehen.
669     Object.values(kapitelRegisterEintraege).forEach(eintrag => eintrag.classList.remove('aktiv'));
670   }
671
672   // Fortschrittsleiste nur ausserhalb einer Kapitel-Ansicht; in der
673   // Graph-Ansicht steht dort der Play-Button.
674   scrollFortschritt.classList.toggle('versteckt', inKapitelAnsicht);
675   grafikPlayButton.classList.toggle('sichtbar', inKapitelGrafikAnsicht);
676   grafikPlayButton.textContent = grafikSpielt ? 'III' : '▶';
677
678   // Kartenbezogene DOM-Overlays blenden sich in der Graph-Ansicht per CSS
679   // aus, siehe .scrolly-stage.grafik-ansicht in style.css.
680   stage.classList.toggle('grafik-ansicht', inKapitelGrafikAnsicht);
681
682   // Stillgelegt, siehe KARTEN_MARKER_SICHTBAR oben.
683   if (KARTEN_MARKER_SICHTBAR) {
684     markierungsintraege.forEach(m => {
685       let p = lonLatToScreen(m.lon, m.lat, activeBbox);
686       m.el.style.left = p.x + 'px';
687       m.el.style.top = p.y + 'px';
688       m.el.classList.toggle('sichtbar', KARTEN_MARKER_SICHTBAR);
689     });
690     stationsMarker.forEach(m => {
691       let p = lonLatToScreen(m.lon, m.lat, activeBbox);
692       m.el.style.left = p.x + 'px';
693       m.el.style.top = p.y + 'px';
694       m.el.classList.toggle('sichtbar', KARTEN_MARKER_SICHTBAR);
695     });
696     zwischenMarker.forEach(m => {
697       let p = lonLatToScreen(m.lon, m.lat, activeBbox);
698       m.el.style.left = p.x + 'px';
699       m.el.style.top = p.y + 'px';
700       m.el.classList.toggle('sichtbar', KARTEN_MARKER_SICHTBAR);
701     });
702   }

```

```

703
704 // Hero-Text ausblenden, kubisch – er bleibt dadurch lange stehen.
705 let heroProgress = constrain(map(progress, SCROLL_MEILENSTEINE.heroFadeStart,
SCROLL_MEILENSTEINE.heroFadeEnd, 0, 1), 0, 1);
706 let heroFade = heroProgress * heroProgress * heroProgress;
707 let heroOpacity = 1 - heroFade;
708 heroText.forEach(el => el.style.opacity = heroOpacity);
709
710 // Begleittexte: jedes <p class="begleittext"> blendet in seinem eigenen
711 // data-von/data-bis-Fenster ein und aus. Neue Texte brauchen kein JS.
712 begleitTexte.forEach(el => {
713   let opacity = begleitTextDeckkraft(progress, parseFloat(el.dataset.von),
parseFloat(el.dataset.bis));
714   // Kapitel 1 hat keinen eigenen Einstiegstext – in der Graph-Ansicht
715   // übernimmt der Begleittext dessen Play-Ausblendweg.
716   if (inKapitelGrafikAnsicht && grafikPlayAusblendStart !== null) {
717     opacity = Math.min(opacity, 1 - constrain(
718       map(millis() - grafikPlayAusblendStart, 0, KAPITEL_EINSTIEG_FADE_MS, 0, 1), 0, 1));
719   }
720   el.style.opacity = opacity;
721 });
722 // Kapitel-Einstiegstexte 02-18: zeitbasierter Fade ab Klick-Zeitpunkt
723 // (kapitelEinstiegsStartMillis, gesetzt von starteKapitelEinstieg oben).
724
725 // Im Übersichtsakt bekommt jedes Kapitel die Scheibe seiner Route: der Text
726 // blendet ein und aus, bevor das nächste an die Reihe kommt.
727 let uebersichtScheiben = !zoomedKapitel ? kapitelScheiben() : null;
728 let imUebersichtsakt = uebersichtRoutenFortschritt > 0
729   && progress < SCROLL_MEILENSTEINE.kreisVergleichStart;
730
731 kapitelEinstiegsTexte.forEach(el => {
732   let passtZuOffenemKapitel = el.dataset.kapitel === zoomedKapitel;
733   let opacity = 0;
734
735   if (uebersichtScheiben && imUebersichtsakt) {
736     let scheibe = uebersichtScheiben.find(sch => sch.nr === el.dataset.kapitel);
737     if (scheibe) {
738       let lokal = constrain(map(uebersichtRoutenFortschritt, scheibe.von, scheibe.bis, 0, 1), 0,
1);
739       // Breites Fenster, damit auch kurze Kapitel mit schmaler Scheibe
740       // lesbar bleiben.
741       opacity = Math.min(
742         constrain(map(lokal, 0, 0.12, 0, 1), 0, 1),
743         1 - constrain(map(lokal, 0.72, 0.88, 0, 1), 0, 1));
744     }
745   }
746   if (passtZuOffenemKapitel && kapitelEinstiegsStartMillis !== null) {
747     let elapsed = millis() - kapitelEinstiegsStartMillis;
748     let einblenden = constrain(map(elapsed, 0, KAPITEL_EINSTIEG_FADE_MS, 0, 1), 0, 1);
749     let ausblenden = 1 - constrain(map(uebersichtRoutenFortschritt,
KAPITEL_EINSTIEG_SCROLL_START, KAPITEL_EINSTIEG_SCROLL_ENDE, 0, 1), 0, 1);
750     // Graph-Ansicht: zusätzlicher Ausblendweg ab dem Play-Klick, gleiche
751     // Dauer wie das Einblenden.
752     let ausblendenPlay = grafikPlayAusblendStart === null ? 1 :
753       1 - constrain(map(millis() - grafikPlayAusblendStart, 0, KAPITEL_EINSTIEG_FADE_MS, 0, 1), 0,
1);
754     opacity = Math.min(einblenden, ausblenden, ausblendenPlay) * kapitelZoomAmount * (1 -
kreisVergleichMapFade);
755   }
756   el.style.opacity = opacity;
757 });
758 // Foto-Marker ganz zuletzt. Offset wie die sichtbare Karte: fixer
759 // mapOffsetX/Y bei gezoomtem Kapitel, sonst kartenOffsetX/Y.
760 let fotoOffsetX = (zoomedKapitel && kapitelZoomAmount > 0.001) ? mapOffsetX : kartenOffsetX;
761 let fotoOffsetY = (zoomedKapitel && kapitelZoomAmount > 0.001) ? mapOffsetY : kartenOffsetY;
762 merkeKartenlage(activeBbox, fotoOffsetX, fotoOffsetY);
763

```

```

764 // Der Bedienhinweis teilt sich das Scroll-Fenster mit seinem Kommentartext,
765 // damit beide zusammen erscheinen und wieder gehen.
766 let fotoHinweis = fotoHinweisText ? {
767   titel: fotoHinweisText.dataset.fotoHinweis,
768   text: FOTO_HINWEIS_TEXT,
769   alpha: begleittextDeckkraft(progress,
770     parseFloat(fotoHinweisText.dataset.von), parseFloat(fotoHinweisText.dataset.bis)),
771 } : null;
772 // In der Graph-Ansicht nicht zeichnen, sonst schweben sie über der Spine.
773 if (!inKapitelGrafikAnsicht) zeichneFotoMarker(activeBbox, fotoOffsetX, fotoOffsetY, 1 -
kreisVergleichMapFade, fotoHinweis);
774
775 // Demo-Kreisgrafik: wächst über den Erklärungstexten heran, schrumpft mit
776 // dem Zoom auf den Icon-Platz oben rechts und bleibt dort stehen. Ganz
777 // zuletzt gezeichnet, damit das Icon über allen Ansichten liegt – auch
778 // über der Graph-Ansicht, die den Rest des Frames überdeckt.
779 let demoFortschritt = constrain(map(progress, SCROLL_MEILENSTEINE.demoStart,
SCROLL_MEILENSTEINE.demoVoll, 0, 1), 0, 1);
780 let demoIkon = constrain(map(progress, SCROLL_MEILENSTEINE.zoomStart, SCROLL_MEILENSTEINE.zoomEnd,
0, 1), 0, 1);
781 // Der Schleier ist vor dem Kreis da: er steigt an, während nur der
782 // Mittelpunkt mit seiner Ortsbeschriftung dasteht, und geht erst nach dem
783 // letzten Legendenschritt wieder weg.
784 let legendeSchleier = Math.min(
785   constrain(map(progress, SCROLL_MEILENSTEINE.kartenwechselEnd, SCROLL_MEILENSTEINE.demoStart, 0,
1), 0, 1),
786   constrain(map(progress, SCROLL_MEILENSTEINE.legendeSchleierAus, SCROLL_MEILENSTEINE.legendeEnde,
1, 0), 0, 1));
787 if (legendeSchleier > 0) zeichneSchleier(LEGENDE_SCHLEIER, LEGENDE_SCHLEIER_ALPHA *
legendeSchleier);
788
789 // Monoton: jeder Schritt blendet mit seinem Erklärungstext ein und bleibt
790 // dann stehen. Der Schleier geht getrennt davon in zeichneDemoKreisgrafik
791 // ein – er nimmt am Ende die Beschriftungen mit, nicht aber die
792 // Differenzierung des Kreises. Sonst fiel das Icon oben rechts auf den
793 // schlichten Streifenkreis der ersten Stufe zurück.
794 let legendeSchritte = demoGruppenTexte.map(el =>
795   legendenSchrittDeckkraft(progress, parseFloat(el.dataset.von), parseFloat(el.dataset.bis)));
796
797 // Beide Ebenen unter die Icons, aber über alles andere: die Icons bleiben
798 // sichtbar, weil sie als Nächstes gezeichnet werden.
799 if (kreisErklaerungOffen) zeichneKreisErklaerung();
800 else if (projekttextOffen) zeichneSchleier(PROJEKTTTEXT_SCHLEIER, PROJEKTTTEXT_SCHLEIER_ALPHA);
801 document.body.classList.toggle('erklaerung-offen', kreisErklaerungOffen);
802 document.body.classList.toggle('projekttext-offen', projekttextOffen);
803 projekttextEl.classList.toggle('offen', projekttextOffen);
804 einblenderSchliessenEl.classList.toggle('sichtbar', kreisErklaerungOffen || projekttextOffen);
805
806 // Ruhezustand beider Icons ist schwarzgrau, erst der Zeiger holt das Gold.
807 // Auf dem dunklen Schleier des Projekttextes ginge Schwarzgrau unter – dort
808 // zeigt die Kreisgrafik ihre echten Farben, das Textzeichen wird hell.
809 let legendeIkonHover = demoIkonGetroffen(mouseX, mouseY);
810 let projekttextIkonHover = projekttextIkonGetroffen(mouseX, mouseY);
811 zeichneDemoKreisgrafik(demoFortschritt, demoAlpha, legendeSchritte, legendeSchleier, demoIkon,
812   demoIkon > 0.99 && !legendeIkonHover && !projekttextOffen);
813 zeichneProjekttextIkon(demoAlpha * demoIkon, projekttextIkonHover, projekttextOffen);
814 // Nach zeichneUebersichtsrouten, das den Cursor jeden Frame selbst setzt.
815 if (legendeIkonHover || projekttextIkonHover) cursor(HAND);
816 }
817
818 // -----
819 // Klicks auf dem Canvas
820 // -----
821 // sonifikation.js liefert nur den Ton, das Bild ist die Graph-Ansicht.
822 // Play-Schalter und Animationsdauer liegen in spine-horizontal.js.
823

```

```

824 function mousePressed() {
825   // Zuerst die beiden Icons: ein Klick darauf schaltet direkt um, statt nur
826   // den offenen Einblender zu schliessen. Die beiden schliessen sich aus.
827   if (demoIkonGetroffen(mouseX, mouseY)) {
828     let aufmachen = !kreisErklaerungOffen;
829     if (projekttextOffen) schliesseProjekttext();
830     kreisErklaerungOffen = aufmachen;
831     return;
832   }
833   if (projekttextIkonGetroffen(mouseX, mouseY)) {
834     kreisErklaerungOffen = false;
835     if (projekttextOffen) schliesseProjekttext();
836     else projekttextPerIkon = true;
837     return;
838   }
839   // Sonst fängt ein offener Einblender jeden Klick ab.
840   if (projekttextOffen) { schliesseProjekttext(); return; }
841   if (kreisErklaerungOffen) { kreisErklaerungOffen = false; return; }
842   if (kapitelHover === '01') { scrolleZuKapitel1(); return; }
843   // ACHTUNG über springeZuKapitelZoom, nicht direkt über oeffneKapitelZoom:
844   // die Startpunkte sind erst weit im Akt sichtbar, ein Klick dort öffnete
845   // das Kapitel sonst mitten im Ablauf.
846   if (kapitelHover) { springeZuKapitelZoom(kapitelHover); return; }
847   if (!letzteActiveBbox) return;
848   // Foto-Marker: Treffertest und Popup liegen in fotomarker.js
849   for (let f of fotoMarkerListe) {
850     let pos = lonLatToScreen(f.lon, f.lat, letzteActiveBbox, letzterFotoOffsetX,
851     letzterFotoOffsetY);
852     if (dist(mouseX, mouseY, pos.x, pos.y) < FOTO_MARKER_TREFFER_RADIUS) {
853       if (f.fotoUrl) oeffneFotoPopup(f);
854       return;
855     }
856   }
857 }
858 // kreisvergleich-orte.json wird vom Code nicht gelesen, bleibt aber als
859 // Prüf-Artefakt: baue-sammelpunkte-handkuriert.py vergleicht dagegen.
860
861
862
863
864 // --- Export -----
865 // 28 Namen: 11 als Wert, 5 p5-Hooks, 12 als Lesebindung.
866
867 // Konstanten, Funktionen und die vier nur befüllten Container: Die Bindung
868 // ändert sich nie, deshalb Wertzuweisung.
869 window.WEITERE_KAPITEL_NUMMERN = WEITERE_KAPITEL_NUMMERN;
870 window.KAPITEL_EINSTIEG_SCROLL_ENDE = KAPITEL_EINSTIEG_SCROLL_ENDE;
871 window.kapitelRegisterEintraege = kapitelRegisterEintraege;
872 window.markierungsEintraege = markierungsEintraege;
873 window.stationsMarker = stationsMarker;
874 window.zwischenMarker = zwischenMarker;
875 window.datenFuerKapitel = datenFuerKapitel;
876 window.kapitelHatEigeneAnsicht = kapitelHatEigeneAnsicht;
877 window.setzeAnsichtsModus = setzeAnsichtsModus;
878 window.loeseKapitel1Klemme = loeseKapitel1Klemme;
879 window.starteKapitelEinstieg = starteKapitelEinstieg;
880
881 // Die fünf p5-Hooks. Nicht optional: p5 sucht sie am window. Fehlt einer,
882 // bleibt das Bild schwarz, ohne Fehlermeldung.
883 window.preload = preload;
884 window.setup = setup;
885 window.draw = draw;
886 window.mousePressed = mousePressed;
887 window.windowResized = windowResized;
888

```



```
889 // Lesebindung für alles, was erst nach dem IIFE-Lauf gesetzt wird – eine
890 // Wertkopie wäre hier durchweg undefined.
891 function lesebindung(name, lies) {
892   Object.defineProperty(window, name, { get: lies, configurable: true });
893 }
894 lesebindung('stationenData', () => stationenData);
895 lesebindung('uebersichtsRouten', () => uebersichtsRouten);
896 lesebindung('kapitelAnsichtsModus', () => kapitelAnsichtsModus);
897 lesebindung('kapitel1ZoomAmount', () => kapitel1ZoomAmount);
898 lesebindung('annotationText', () => annotationText);
899 lesebindung('kartenMarkierungenEl', () => kartenMarkierungenEl);
900 lesebindung('kapitelRegister', () => kapitelRegister);
901 lesebindung('modusZeile', () => modusZeile);
902 lesebindung('planEintrag', () => planEintrag);
903 lesebindung('graphEintrag', () => graphEintrag);
904 lesebindung('leerzeile', () => leerzeile);
905 lesebindung('alleEintrag', () => alleEintrag);
906
907 }()); // Ende der Modulkapselung, siehe Kommentar oben
```